6.1

25 de septiembre de 2026

6.1 - Cài đặt giải thuật Apriori từ đầu (Scratch)

Họ và tên: Thái Anh Lạc | **MSSV:** 089206000860 | **Mã lớp HP:** 012012204103

In [1]:

```
# 1. Khởi tạo dữ liệu giao dịch Market-Basket
dataset = [
    ['Bread', 'Milk'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Coke'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Coke']
]

# Chọn tham số phù hợp:
# - minSup = 0.4 (40%, tối thiểu 2/5 giao dịch để trích xuất được tới 3-itemset)
# - minConf = 0.6 (60% độ tin cậy)
N = len(dataset)
min_sup = 0.4
min_sup_count = min_sup * N
min_conf = 0.6

print(f"Số lượng giao dịch: {N}")
print(f"Tham số chọn: minSup = {min_sup} (tối thiểu {int(min_sup_count)}/{N} giao dịch), minConf = {min_conf} ({int(min_conf*100)}
```

Out [1]:

```
Số lượng giao dịch: 5
Tham số chọn: minSup = 0.4 (tối thiểu 2/5 giao dịch), minConf = 0.6 (60%)
```

In [2]:

```
# 2. Cài đặt các hàm giải thuật Apriori
from itertools import combinations

def Apriori_prune(Ck, minsupport):
    """Lọc các itemset có tần suất >= minsupport"""
    L = []
    for i in Ck:
        if Ck[i] >= minsupport:
            L.append(i)
    return sorted(L)

def Apriori_gen(Itemset, length):
    """Sinh ứng viên (k+1)-itemset từ k-itemset"""
    candidate = []
    for i in range(length):
        element = Itemset[i]
        for j in range(i + 1, length):
            element1 = Itemset[j]
            # Kết nối nếu k-2 phần tử đầu giống nhau
            if element[:-1] == element1[:-1]:
                unionset = tuple(sorted(set(element) | set(element1)))
                if unionset not in candidate:
                    candidate.append(unionset)
    return candidate

def Apriori_count_subset(Candidate, dataset):
    """Đếm số lần xuất hiện của các ứng viên trong tập giao dịch"""
    Lk = {can: 0 for can in Candidate}
    for trans in dataset:
        trans_set = set(trans)
        for can in Candidate:
            if set(can).issubset(trans_set):
                Lk[can] += 1
```

```
    return Lk

def generate_rules(all_frequent, all_counts, N, min_conf):
    """Sinh các luật kết hợp thỏa mãn minConf"""
    rules = []
    for k, itemsets in all_frequent.items():
        if k < 2:
            continue
        for itemset in itemsets:
            items = list(itemset)
            for i in range(1, len(items)):
                for antecedent in combinations(items, i):
                    antecedent = tuple(sorted(antecedent))
                    consequent = tuple(sorted(set(itemset) - set(antecedent)))
                    sup = all_counts[itemset] / N
                    conf = all_counts[itemset] / all_counts[antecedent]
                    if conf >= min_conf:
                        rules.append((antecedent, consequent, sup, conf))
    return rules
```

In [3]:

```
# 3. Chạy thuật toán tìm các tập phổ biến (Frequent Itemsets)
# Đếm 1-itemset (C1)
C1 = {}
for line in dataset:
    for item in line:
        key = (item,)
        C1[key] = C1.get(key, 0) + 1

all_counts = dict(C1)
all_frequent = {}

# Lọc L1
L1 = Apriori_prune(C1, min_sup_count)
all_frequent[1] = L1

print("====================================")
print(f"Frequent 1-itemset ({len(L1)} tập):")
for item in L1:
    sup = C1[item] / N
    print(f"  {set(item)} | Support = {sup:.2f} ({C1[item]}/{N})")
print("====================================")

# Lặp sinh Lk với k >= 2
L = Apriori_gen(L1, len(L1))
k = 2
while L:
    C = Apriori_count_subset(L, dataset)
    all_counts.update(C)
    frequent_itemset = Apriori_prune(C, min_sup_count)
    if not frequent_itemset:
        break
    all_frequent[k] = frequent_itemset
    print(f"Frequent {k}-itemset ({len(frequent_itemset)} tập):")
    for item in frequent_itemset:
        sup = C[item] / N
        print(f"  {set(item)} | Support = {sup:.2f} ({C[item]}/{N})")
    print("====================================")
    L = Apriori_gen(frequent_itemset, len(frequent_itemset))
    k += 1
```

Out [3]:

```
====================================
Frequent 1-itemset (5 tập):
  {'Beer'} | Support = 0.60 (3/5)
  {'Bread'} | Support = 0.80 (4/5)
  {'Coke'} | Support = 0.40 (2/5)
  {'Diaper'} | Support = 0.80 (4/5)
  {'Milk'} | Support = 0.80 (4/5)
====================================
Frequent 2-itemset (8 tập):
  {'Beer', 'Bread'} | Support = 0.40 (2/5)
```

```
  {'Beer', 'Diaper'} | Support = 0.60 (3/5)
  {'Beer', 'Milk'} | Support = 0.40 (2/5)
  {'Bread', 'Diaper'} | Support = 0.60 (3/5)
  {'Bread', 'Milk'} | Support = 0.60 (3/5)
  {'Coke', 'Diaper'} | Support = 0.40 (2/5)
  {'Coke', 'Milk'} | Support = 0.40 (2/5)
  {'Milk', 'Diaper'} | Support = 0.60 (3/5)
==================================
Frequent 3-itemset (4 tập):
  {'Beer', 'Bread', 'Diaper'} | Support = 0.40 (2/5)
  {'Beer', 'Milk', 'Diaper'} | Support = 0.40 (2/5)
  {'Bread', 'Milk', 'Diaper'} | Support = 0.40 (2/5)
  {'Milk', 'Coke', 'Diaper'} | Support = 0.40 (2/5)
==================================
```

In [4]:

```
# 4. Sinh và xuất các luật kết hợp (Association Rules)
rules = generate_rules(all_frequent, all_counts, N, min_conf)

# Sắp xếp các luật theo Confidence giảm dần, sau đó theo Support
sorted_rules = sorted(rules, key=lambda x: (x[3], x[2]), reverse=True)

print(f"DANH SÁCH CÁC LUẬT KẾT HỢP (minConf = {min_conf}, minSup = {min_sup}) - Tổng: {len(sorted_rules)} luật:\n")
for idx, (ant, con, sup, conf) in enumerate(sorted_rules, 1):
    ant_str = str(set(ant))
    con_str = str(set(con))
    print(f"{idx:2d}. {ant_str:<25} => {con_str:<25} | Support = {sup:.2f} ({sup*100:.0f}%) | Confidence = {conf:.2f} ({conf*100:.
```

Out [4]:

```
DANH SÁCH CÁC LUẬT KẾT HỢP (minConf = 0.6, minSup = 0.4) - Tổng: 27 luật:

 1. {'Beer'}                  => {'Diaper'}                | Support = 0.60 (60%) | Confidence = 1.00 (100.0%)
 2. {'Coke'}                  => {'Diaper'}                | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 3. {'Coke'}                  => {'Milk'}                  | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 4. {'Beer', 'Bread'}         => {'Diaper'}                | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 5. {'Beer', 'Milk'}          => {'Diaper'}                | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 6. {'Coke'}                  => {'Milk', 'Diaper'}        | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 7. {'Coke', 'Diaper'}        => {'Milk'}                  | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 8. {'Coke', 'Milk'}          => {'Diaper'}                | Support = 0.40 (40%) | Confidence = 1.00 (100.0%)
 9. {'Diaper'}                => {'Beer'}                  | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
10. {'Bread'}                 => {'Diaper'}                | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
11. {'Diaper'}                => {'Bread'}                 | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
12. {'Bread'}                 => {'Milk'}                  | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
13. {'Milk'}                  => {'Bread'}                 | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
14. {'Diaper'}                => {'Milk'}                  | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
15. {'Milk'}                  => {'Diaper'}                | Support = 0.60 (60%) | Confidence = 0.75 (75.0%)
16. {'Beer'}                  => {'Bread'}                 | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
17. {'Beer'}                  => {'Milk'}                  | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
18. {'Beer'}                  => {'Bread', 'Diaper'}       | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
19. {'Beer', 'Diaper'}        => {'Bread'}                 | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
20. {'Bread', 'Diaper'}       => {'Beer'}                  | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
21. {'Beer'}                  => {'Milk', 'Diaper'}        | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
22. {'Beer', 'Diaper'}        => {'Milk'}                  | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
23. {'Milk', 'Diaper'}        => {'Beer'}                  | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
24. {'Bread', 'Diaper'}       => {'Milk'}                  | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
25. {'Bread', 'Milk'}         => {'Diaper'}                | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
26. {'Milk', 'Diaper'}        => {'Bread'}                 | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
27. {'Milk', 'Diaper'}        => {'Coke'}                  | Support = 0.40 (40%) | Confidence = 0.67 (66.7%)
```